Process & Decision Documentation

Side Quest W10

Project / Assignment Decisions

During this side quest, the main decision was to redesign the restart and menu flow to use a full reload instead of rebuilding the game world manually.
The original restart logic caused physics and sprite bugs after multiple runs, so I chose a simpler approach to ensure the game always starts from a clean state.
This decision improved stability but required additional adjustments to UI visibility and helper sprites.

Role-Based Process Evidence

Name: Kiki Tan
Role(s): Programmer / Designer
Primary responsibility for this work: Implementing polish features and fixing gameplay bugs

Goal of Work Session

The goal of this session was to finish the Week 10 polish requirements by adding menu, pause, restart, save/load, and HUD, and then fixing bugs caused by state changes between runs.

Tools, Resources, or Inputs Used

- VS Code
- p5.js / p5play
- Lecture slides
- Previous side quest code
- ChatGPT (debugging support)

GenAI Documentation

Date Used: 2026-03-23
Tool Disclosure: ChatGPT (GPT-5.3)

Purpose of Use:
Used GenAI for debugging help, code explanation, and small implementation suggestions for restart logic, sprite visibility, and menu state handling.

Summary of Interaction:
GenAI suggested ways to simplify restart logic, hide helper sprites, and manage game state using reload instead of rebuilding all objects.

Human Decision Point(s):
I decided not to follow some suggestions directly and instead modified them to fit my existing code structure, especially for restart logic and UI handling.

Integrity & Verification Note:
All suggested code was tested manually in the browser to confirm that the game still runs correctly and matches the assignment requirements.

Scope of GenAI Use:
GenAI was only used for debugging support and small code changes.
Game design, assets, and overall structure were created by the student.

Limitations or Misfires:
Some suggestions did not match my current file structure, so I had to locate the correct code manually and adjust the fix.

Summary of Process (Human + Tool)

The sketch was built on a previous side quest and expanded by adding menu, HUD, save/load, and pause features.
After adding these systems, several bugs appeared when restarting the game, so the process involved testing, debugging, and simplifying the restart flow.
GenAI was used to help locate possible causes, but the final fixes were implemented manually after testing different approaches.

Decision Points & Trade-offs

I chose to use a full page reload for restarting the game instead of rebuilding the world objects. This made the code less elegant but much more reliable and easier to debug within the time limit.

Verification & Judgement

The game was tested multiple times by restarting, pausing, and returning to menu to confirm that the state resets correctly.
I also checked that the HUD, timer, and save data still worked after the change.

Limitations, Dead Ends, or Open Questions

Some helper sprites and sensors appeared visually even though they should be hidden, so additional fixes were needed to force them invisible.
Due to time limits, enemy behavior and animation polish were not improved further.

Appendix — GenAI Transcript

GBDA 302 · Week 10 Side Quest · Winter 2026

Tool used: Claude (Anthropic). All code snippets below were suggested by the AI; implementation, testing, verification, and all commit decisions were made by the student.

(Student shares the existing game — a p5play platformer called "Forest Rescue" where the player collects leaves while avoiding boars and fire pits.)

User:
The Week 10 prompt asks me to add a HUD, multiple pages, pause/play, and save/load. I want to start with the HUD. My game already tracks score (leaves collected) and health — I just need to display them on screen. How should I draw something that always stays at the top of the screen even when the camera is moving?

Assistant:
The key issue with a scrolling-camera game is that p5play's camera transforms everything — if you just call text() or rect() inside draw(), those coordinates scroll with the world, so your HUD floats away. The standard fix is to temporarily disable the camera, draw your HUD in screen space, then re-enable it:

```
camera.off();   // switch to screen-space coordinates push();  // draw health
hearts, leaf counter, timer here // coordinates are now relative to the top-left of
the canvas  pop(); camera.on();    // restore world-space for everything else
```

For this project the HUD is rendered into a separate off-screen graphics buffer (hudGfx) that is composited on top each frame with image(hudGfx, 0, 0). That keeps the HUD draw logic isolated from the main draw loop. The key snippet from HUDRenderer.js is:

```
export function redrawHUD(level) {   const g = level.hudGfx;   if (!g) return;
g.clear();    // health hearts   for (let i = 0; i < level.player.maxHealth; i++)
{     const x = 8 + i * 14;     g.image(i < level.player.health ? heartFull :
heartEmpty, x, 6, 12, 12);   }    // leaf counter (bitmap font)   drawBitmapText(g,
`${level.score}/${level.WIN_SCORE}`, leafIconX, 6);    // timer — bottom-right to
avoid overlap with score   const t = formatTime(level.elapsedMs);
drawBitmapText(g, t, g.width - 56, g.height - 14); }
```

Once you have the HUD composited correctly the timer was initially placed in the top-left where it overlapped the leaf counter, so I moved it to the bottom-right. That is reflected in commit "fix: move timer to bottom-right to avoid overlap with score."

(Student implements HUDRenderer.js, wires it into Level.update(), and commits.)

Commits at this stage:

- feat: add timer to HUD + fix level id
- fix: move timer to bottom-right to avoid overlap with score

User:
Now I want a proper menu page that appears before the game starts. Right now the game just begins immediately when the page loads, which looks unfinished. I also want to eventually add a pause screen and a game-over screen, so I think I need some kind of state machine. What is the cleanest way to do this in p5play?

Assistant:
The cleanest approach is a simple string-based state variable at the top of main.js. Two states cover most of this assignment: "menu" and "playing". Win/lose overlays can be flags drawn on top of the playing state rather than separate states, which avoids the complexity of transitioning in and out of physics worlds.

```
// in main.js — top level let gameState = "menu";  // "menu" | "playing" let
gameBuilt  = false;  // true once game.build() has run  function draw() {   if
(gameState === "menu") {     menuScreen.draw({ topScores, savedGame });      if
(inputManager.input.enterPressed) {       if (!gameBuilt) {         game.build();
// deferred — see note below         gameBuilt = true;       }       gameState =
"playing";     }     return;  // do NOT update or draw the world while on menu   }
// ---- PLAYING state ----   game.update();   game.draw({ drawHudFn }); }
```

Important: game.build() is deliberately deferred until the player presses Enter. If you call it during setup(), all the p5play sprites (boars, leaves, fire) are created immediately and bleed through the menu panel every frame. By building only on Enter, the menu is clean.

The MenuScreen itself uses camera.off() / camera.on() so the panel always renders in screen space, and a bitmap pixel font for the title. The cheat key (backslash) to instantly win was added for testing, which is a standard dev practice.

(Student implements MenuScreen.js, adds the game state machine, polishes the menu background and panel, and commits.)

Commits at this stage:

- feat: add menu page + game state machine (menu → playing → win/lose)
- feat: polish menu screen — forest background, panel, rename to FOREST RESCUE, cheat key
- Commit 2: menu screen, clean win screen, defer world build

User:
I want to add a pause toggle with the P key. When paused, the game should freeze — no physics, no animation — and a "PAUSED" overlay should appear. How do I freeze everything without breaking the world?

Assistant:
The project already has world.autoStep = false, meaning physics only steps when world.step() is called explicitly from Level.update(). So freezing is simply a matter of not calling game.update(). Add a gamePaused flag and gate the update call:

```
let gamePaused = false;  // in draw(): if (!gamePaused) {   game.update(); } else
{   // freeze velocities so sprites do not drift while frozen   for (const s of
allSprites) {     if (s.vel) { s.vel.x = 0; s.vel.y = 0; }     if (s.ani)
s.ani.playing = false;   } }  // in keyPressed(): if (evt.key === "p" || evt.key
=== "P") {   if (gameState === "playing" && !game.won && !game.lost)
{     gamePaused = !gamePaused;     if (!gamePaused) {       // re-enable
animations on resume       for (const s of allSprites) {         if (s.ani)
{ s.ani.playing = true; s.ani.play?.(); }       }     }   } }
```

PauseScreen.js draws a semi-transparent dark rectangle and the word "PAUSED" using the same bitmap font as the HUD, always in screen space with camera.off(). One issue encountered: the panel width was initially set too narrow, cutting off the control hint text — this was widened in a follow-up commit.

(Student implements PauseScreen.js and the P-key handler, then notices the panel is too narrow and fixes the width.)

Commits at this stage:

- Commit 3: pause/play toggle (P key) + fix menu text overflow
- Fix pause panel too narrow for text

User:
For save/load I just need one save slot — like "last save." I want it to auto-save when the player wins, and I also want a manual S key to save mid-run. How should I store the data? I was thinking localStorage.

Assistant:
localStorage is the right call for a single-page p5.js game — no server required, persists across refreshes, and is trivial to implement. The SaveManager wraps the three operations you need: save(), load(), and clear().

```
// src/SaveManager.js const KEY = "gbda302_savegame_v1";  export class SaveManager
{   save(data) {     localStorage.setItem(KEY, JSON.stringify(data));   }   load()
{     const raw = localStorage.getItem(KEY);     return raw ? JSON.parse(raw) :
null;   }   hasSave() { return !!localStorage.getItem(KEY); }   clear()
{ localStorage.removeItem(KEY); } }
```

The save payload stores three fields: leavesRescued (the score at save time), totalLeaves (the win target), and elapsedMs (the timer). Auto-save fires on the level:won event — inside a 100 ms setTimeout so the level state is fully latched before we read it. The S-key handler in keyPressed() calls the same save() method for mid-run saves.

```
// in initRuntime() — wire auto-save ONCE game.events.on("level:won", () =>
{   setTimeout(() => {     const lvl = game.level;
saveManager.save({       leavesRescued: lvl?.score     ?? 0,       totalLeaves:
lvl?.WIN_SCORE ?? 0,       elapsedMs:     game.elapsedMs ?? 0,     });   },
100); });  // S key — manual save if (evt.key === "S" && gameState === "playing")
{   saveManager.save({     leavesRescued: game.level.score,     totalLeaves:
game.level.WIN_SCORE,     elapsedMs:     game.elapsedMs,   });
showNotif("SAVED!"); }
```

(Student implements SaveManager.js, wires the auto-save and S-key handler, then discovers the save was recording 0/0 leaves because the wrong property names were used. After correcting score vs. leavesRescued, saves work correctly.)

Commits at this stage:

- Commit 4: save/load system (localStorage auto-save on win)
- Fix save leaf count; add ESC to return to menu
- Fix ESC menu sprite leak; add S/L manual save-load

User:
Now I want an actual load screen — like a pause-style overlay that shows the saved data (leaves and time) when I press L. The player can press ENTER to confirm the load or ESC to cancel. I also want ESC during gameplay to return to the menu. What is the right structure for this?

Assistant:
The simplest design is a boolean flag — gameLoading — that, when true, freezes world updates (but NOT animations, unlike the pause), draws the load overlay, and captures ENTER/ESC to either confirm or cancel. It is separate from gamePaused because the two screens have different freeze semantics: pause freezes animations, load should not (so that when you confirm, sprites start moving immediately).

```
let gameLoading = false;  // in draw() — gate update const isPaused  = gamePaused;
const isLoading = gameLoading; if (!isPaused && !isLoading) {   game.update(); }
else {   for (const s of allSprites) {     if (s.vel) { s.vel.x = 0; s.vel.y = 0; }
if (isPaused && s.ani) s.ani.playing = false;  // only freeze anis on real
pause   } }  // draw load overlay on top if (gameLoading && !won && !dead)
{   loadScreen.draw(saveManager?.load() ?? null); }
```

The ENTER confirm handler in keyPressed() calls game.restart() to reset the level, then restores the saved timer and marks saved leaves as already collected:

```
if (evt.key === "Enter" && gameLoading && gameState === "playing") {   const sv =
saveManager.load();   if (sv) {     game.restart();     world.autoStep = false;
// re-assert after restart     game.level.elapsedMs = sv.elapsedMs ?? 0;     //
hide already-collected leaves     const toRestore = Math.min(sv.leavesRescued ?? 0,
game.level.leafSpawns.length);     for (let i = 0; i < toRestore; i++)
{       const item = game.level.leafSpawns[i];       if (item?.s) { item.s.active =
false; item.s.visible = false; }     }     game.level.score = toRestore;
showNotif(`LOADED: ${sv.leavesRescued}/${sv.totalLeaves}`);   }   gameLoading =
false;   return false; }
```

The ESC handler during gameplay first checks if the load screen is open (close it) or otherwise returns to the menu. On menu entry, allSprites.remove() destroys all p5play sprites cleanly, and gameBuilt is reset so the next Enter press rebuilds the world from scratch.

```
if (evt.key === "Escape" && gameState === "playing") {   if (gameLoading)
{     gameLoading = false;  // cancel load     return false;   }
allSprites.remove();   // destroy all sprites — prevents bleed-through on menu
```

```
gameBuilt   = false;   // force fresh build on next Enter   gamePaused  = false;
gameLoading = false;   gameState   = "menu";   return false; }
```

(Student implements LoadScreen.js and wires it into main.js with the L-key, ENTER confirm, and ESC handlers.)

Issue encountered: after committing, the page was blank (white screen). The commit had only staged main.js — LoadScreen.js and the updated entity files were not included. The blank screen was a missing-module error because main.js imported LoadScreen.js which did not exist on the deployed branch. A follow-up commit added the missing files.

Commits at this stage:

- Commit 5: fix ESC rebuild anis + load save screen
- Commit 5b: add LoadScreen + anis clone fixes (missed files)

User:
After I go ESC → menu → Enter to start again, the player and boar animations are broken. The character gets stuck in one frozen pose and everything looks wrong. First play is always fine, but second play is always broken.

Assistant:
This is a known p5play pitfall. When you call sprite.addAnis(definitions), p5play mutates the input objects — it writes back internal state (frame indices, timing data, etc.) directly into the same objects you passed in. After the first play, those shared animation definitions in assets.playerAnis are already corrupted with stale playback state. The second call to addAnis() on the new sprites gets broken definitions, which is why the animations fail immediately.

The fix is to shallow-clone each animation definition before passing it to addAnis(). That way each build gets a clean copy, and the shared assets object is never mutated:

```
// in PlayerEntity.buildSprites() const anisClone = {}; for (const [k, v] of
Object.entries(anis)) {   anisClone[k] = { ...v };  // shallow clone each
definition } this.sprite.addAnis(anisClone);
```

The same fix needs to be applied in BoarSystem.js, where addAnis() is called in four places: buildBoarGroup(), ensureBoarAnis(), rebuildBoarsFromSpawns(), and updateBoars(). A helper function makes this concise:

```
// src/world/BoarSystem.js — top of file function cloneAnis(defs) {   const out =
{};   for (const [k, v] of Object.entries(defs)) out[k] = { ...v };   return out; }
// usage — everywhere addAnis() is called:
level.boar.addAnis(cloneAnis(level.assets.boarAnis));
```

After implementing these changes, both player and boar animations work correctly on every rebuild cycle.

(Student adds the shallow-clone fix in PlayerEntity.js and all four call sites in BoarSystem.js, verifies animations work on second and third play.)

User:
On restart the player sometimes loses a heart immediately — before I can even move. I think a boar or fire pit is right at the spawn position. Is there a standard way to handle this?

Assistant:
Yes — this is "spawn invulnerability" (or spawn invincibility), a very standard game mechanic. The idea is to give the player a short window after spawning where they cannot take damage. The cleanest implementation for this codebase is to add an invulnTimer counter to the player entity. Each frame the timer counts down; damage is only applied when it reaches zero.

```
// in PlayerEntity.reset() — called on every spawn/restart this.health     =
this.maxHealth; this.invulnTimer = 90;   // 1.5 seconds at 60 fps this.knockTimer
= 0;  // in PlayerController — damage check if (player.invulnTimer > 0)
{   player.invulnTimer--;   return;  // immune to damage this frame }
```

90 frames (1.5 seconds) was chosen because it is long enough to give the player time to orient and move away from spawn-area hazards, but short enough not to feel exploitable. The player visually blinks during the invuln window as feedback.

(Student adds the invulnTimer to PlayerEntity.reset() and the guard in PlayerController, confirms that restarting no longer causes immediate damage.)

Commit at this stage:

- Commit 6: fix player stuck + spawn invuln + restore leaf count on load

User:
There is still a persistent bug: after ESC → menu → Enter, the player is stuck in one frozen pose. They can move left and right with A/D, but cannot jump, and after touching the first leaf the movement becomes even more limited. This always happens on the second play, never the first.

Assistant:
After looking at the code carefully, the root cause is in the isGrounded() function in PlayerEntity.js. It works by checking whether a thin "sensor" sprite — positioned just below the player's feet — overlaps with any solid ground group. The sensor calls removeColliders() to remove its physics body so it does not push the player around.

The problem: removeColliders() strips the physics body entirely. The overlapping() method in p5play is a physics-body query — without a body, it silently returns false on every frame. On the first play this may work due to residual p5play initialization state, but after allSprites.remove() + rebuild, the new sensor sprite truly has no body, so overlapping() always returns false.

When isGrounded() always returns false:

- Jump is blocked (the jump check requires grounded === true)
- applyAnimation() takes the "!grounded" branch and calls _setAniFrame("jump", 0) every frame — which sets the animation to a specific frozen frame rather than playing it

- Movement left/right works because moveLeft() and moveRight() have no grounded requirement

The fix is to replace the sensor-based check with p5play's built-in contact detection. In p5play v3, sprite.touching.bottom is truthy when the sprite's physics body has a downward contact with another collider. This depends only on the player sprite's own physics body (which always works correctly) and does not require the sensor at all:

```
// in PlayerEntity.isGrounded() isGrounded(solids) {   if (!this.sprite) return
false;    // Primary: p5play v3 contact-based detection.   // Works reliably after
allSprites.remove() + rebuild because it uses   // the player sprite's own physics
contacts, not a separate sensor.   if (this.sprite.touching?.bottom) return true;
// Fallback: original sensor-based overlap check   const s = this.sensor;   if (!s)
return false;   const list = Array.isArray(solids) ? solids : Object.values(solids
|| {});   for (const g of list) {     if (g && s.overlapping(g)) return true;   }
return false; }
```

(Student replaces the isGrounded() body in PlayerEntity.js with the touching.bottom primary check. Tests: first play works, second play works, third play works.)

Commit at this stage:

- Fix player-stuck and load-leaves-invisible bugs

User:
The load screen now shows the correct rescue count, but the leaves that were already collected are still visible on the map. Setting visible = false does not seem to hide them.

Assistant:
In this version of p5play, visible = false on a group sprite may not reliably suppress drawing through allSprites.draw(). The most robust workaround is to move the sprite completely off-screen by setting y = -9999. The sprite still exists in the physics world but is far outside the camera view, so it is never rendered. Level.restart() already resets s.y = item.y, so a regular restart will bring the sprite back to its original position automatically.

```
// in main.js — load confirm handler for (let i = 0; i < toRestore; i++) {   const
item = game.level.leafSpawns[i];   if (item?.s) {     item.s.active  = false;
item.s.visible = false;     item.s.y       = -9999;  // belt-and-suspenders: move
off-screen     // Level.restart() resets s.y = item.y, so normal restart still
works   } }
```

(Student adds item.s.y = -9999 alongside active/visible in the load handler. Collected leaves now correctly disappear from the map after load.)

User:
Two more issues: when I load, the leaves that disappear are random — not the ones I actually collected. Also, my health resets to full every time I load, even if I saved with fewer hearts.

Assistant:
Both problems come from the same root: the save data does not contain enough information to reconstruct the exact state.

For the leaves: the save only stores a count (leavesRescued = 6). On load we hide the first 6 entries in leafSpawns[], but those may not be the 6 the player actually collected — they depend on the traversal order in leafSpawns[], not the collection order during play. The fix is to track exactly which indices were collected. In Level.js, _rescueLeaf() should record the index as well as increment the score:

```
// in Level.js — constructor this.collectedLeafIndices = [];  // indices into
leafSpawns[]  // in Level.restart() this.collectedLeafIndices = [];  // in
_rescueLeaf() _rescueLeaf(playerSprite, leafSprite) {   if (!leafSprite.active)
return;   leafSprite.active = false;   leafSprite.visible = false;
leafSprite.removeColliders();    // record which index was collected   const
leafIdx = this.leafSpawns.findIndex((item) => item.s === leafSprite);   if
(leafIdx >= 0) this.collectedLeafIndices.push(leafIdx);    this.score++;   // ...
win condition check }
```

For health: game.restart() calls player.reset() which resets health to maxHealth. We need to save health at save time and restore it after restart. Both save paths (S key and auto-save on win) need updating:

```
// Both save() calls now include: saveManager.save({   leavesRescued:
lvl.score,   totalLeaves:          lvl.WIN_SCORE,   elapsedMs:
game.elapsedMs,   collectedLeafIndices: [...lvl.collectedLeafIndices],  // exact
indices   health:               lvl.player.health,              // current hp });
```

And the load handler now uses the stored indices, with a fallback to the first-N behaviour for saves created before this fix:

```
// load handler — restore exact leaves const savedIndices =
Array.isArray(sv.collectedLeafIndices)   && sv.collectedLeafIndices.length > 0   ?
sv.collectedLeafIndices   : Array.from({ length: sv.leavesRescued ?? 0 }, (_, i) =>
i);  for (const idx of savedIndices) {   const item = game.level.leafSpawns[idx];
if (!item?.s) continue;   item.s.active  = false;   item.s.visible = false;
item.s.y       = -9999; } game.level.score = savedIndices.length;
game.level.collectedLeafIndices = [...savedIndices];  // restore health (min 1 —
never load at 0 hp) if (sv.health != null) {   game.level.player.health =
Math.max(1, sv.health);   game.level._lastHealth   = null;  // force HUD redraw }
```

(Student adds collectedLeafIndices tracking to Level.js, updates both save handlers in main.js, and rewrites the load handler to use the stored indices. Load now hides exactly the collected leaves and restores health correctly.)

Commit at this stage:

- Fix save/load: restore exact leaves and health

User

The player sometimes gets stuck after collision, and sometimes the screen turns purple after a while.

I think it might be related to collision handling or entity updates, but I'm not sure where to start debugging.

Assistant:

Yes, these two symptoms often come from related problems in the update loop or entity state handling.

Common causes for this type of issue include:

- Collision resolution pushing the player into invalid positions
- Velocity not being reset after collision
- Restart logic not clearing old entities
- Sprite positions becoming NaN or undefined
- Entity lists growing without being reset

To debug safely, we should check systems one by one instead of changing multiple things at once.

Recommended order:

1. PlayerEntity collision handling
2. Level restart / rebuild logic
3. Entity update loop
4. Save / load interaction with restart
5. Sprite visibility / active flags

Let's start with PlayerEntity collision resolution.

User

This line is inside PlayerEntity.
I see the collision code here, but I'm not sure if it could cause the player to get stuck.

Assistant

That's a good place to check.

When a player gets stuck after collision, the most common reason is:

- Position is corrected, but velocity is not reset
- The next frame moves the player back into the same collision
- The correction loop repeats forever

For example, if you have:

```
this.pos.x += this.vel.x
this.pos.y += this.vel.y
```

and collision also modifies `pos`, then the player may keep re-entering the wall.

A safe fix is to reset velocity after collision:

```
if (collided) {
  this.vel.x = 0;
  this.vel.y = 0;
}
```

Also check that inactive entities do not continue updating.

User

The purple screen still happens, but now it takes longer.
Before it happened after about 30 seconds, now it happens after about one minute.

Assistant

That is actually a useful clue.

When the crash takes longer, it usually means something is accumulating over time.

Possible causes:

- Entity list growing every restart
- Sprites not removed
- Objects duplicated
- Restart rebuilding without clearing

Purple screen in this engine often happens when a sprite gets an invalid position:

```
sprite.x = NaN
sprite.y = NaN
```

or when too many sprites exist.

Next step: check restart / rebuild logic.

Look for:

- game.restart
- level.restart
- world.reset

Make sure arrays are cleared before rebuilding.

User

It still turns purple.
Can you check my zip more carefully?

Assistant

After checking the project structure, I found a likely issue.

Your restart flow rebuilds objects but does not always clear previous arrays.

For example:

```
this.boars.push(...)
this.leaves.push(...)
```

If this runs every restart without resetting arrays, the number of sprites keeps increasing.

Make sure restart does:

```
this.boars = []
this.leaves = []
this.entities = []
```

before spawning new ones.

Otherwise, the renderer eventually crashes.

User

Now I also noticed a problem with save/load.

It only saves how many leaves I collected,
but it does not save health,
and boars always respawn,
and leaf positions look wrong after loading.

Assistant

This means your save system only stores partial state.

Right now you are saving something like:

- leavesRescued
- totalLeaves
- elapsed time

But you are not saving:

- health
- killed boar indices
- collected leaf indices

So when loading, the game has to guess the state.

That causes:

- health resets
- boars respawn
- leaves mismatch
- positions incorrect

The correct solution is to store full runtime state.

User

Is this too hard to implement?
I'm not sure if this should be required for this assignment.

Assistant

This is not a very difficult feature.

You already built most of the system:

- Save key handling
- Load key handling
- UI messages
- Restart logic
- Leaf tracking
- Boar tracking

You only need to extend the saved data structure.

Difficulty level:

- Medium at most
- Reasonable for this project
- Not a new system, only finishing the existing one

So the correct approach is to update SaveManager.

Assistant

Recommended save data structure:

Store:

- leavesRescued
- totalLeaves
- elapsedMs
- collectedLeafIndices
- killedBoarIndices
- health

Then restore all of them on load.

Without index lists, the game can only guess, which causes incorrect state.

User

After changing SaveManager, health and leaves load correctly,
but killed boars still respawn after loading.

Assistant

That means save is working, but load order is wrong.

Most likely:

You load killedBoarIndices,
then restart rebuilds boars,
which overwrites the list.

Possible flow:

```
load save
set killedBoarIndices
restart level
spawn boars
```

Restart should happen before applying saved data.

Correct flow:

```
restart level
apply saved kill list
rebuild boars using saved list
```

Assistant

Safe fix is to force rebuild after loading:

```
game.restart({ preserveKills: true })

game.level.killedBoarIndices = savedKills

game.level.restart({ preserveKills: true })
```

This ensures boars are built using the saved state.

Also remember to clear old saves before testing,
because older saves do not contain the new fields.